

Real-Time Programming

Raghav Vinjamuri

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

Real-Time Programming

- interaction between a computer system and the real world happens in real time, and so this is an important topic for developers of embedded systems.
- There are problems such as scheduling policies and priority inversion; page faults and memory locking.
- So the author brings these topics together and look at real-time programming in some depth.
 - What are the characteristics of realtime systems, and considerations for system design, at both the application and kernel levels.
 - What is the real-time PREEMPT_RT kernel patch, and how to get it and apply it to a mainline kernel.
 - describe how to characterize system latencies using two tools: cyclicttest and Ftrace.
- There are other ways to achieve real-time behavior on an embedded Linux device, for instance, using a dedicated microcontroller or a separate real-time kernel alongside the Linux kernel e.g. Xenomai and RTAI
- They are not discussed these here because the focus of this book is on using Linux as the core for embedded systems.
- What is real time?
- Identifying sources of non-determinism
- Understanding scheduling latency
- Kernel preemption
- The real-time Linux kernel (PREEMPT_RT)
- Preemptible kernel locks
- High-resolution timers
- Avoiding page faults
- Interrupt shielding
- Measuring scheduling latencies

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Chapter Objectives

In This Chapter

- Real-Time Programming
- Assignment
- Quiz

CHAPTER OVERVIEW

In This Chapter, We Will Discuss

1. Real-Time Programming

What is real time?

Identifying sources of non-determinism

Understanding scheduling latency, Kernel preemption

The real-time Linux kernel (PREEMPT_RT)

Preemptible kernel locks

High-resolution timers

Avoiding page faults

Interrupt shielding

Measuring scheduling latencies

install the latest LTS release of Buildroot for

- If not yet installed, then refer to the earlier notes and System requirements section of The Buildroot user manual (<https://buildroot.org/downloads/manual/manual.html>).

install the latest LTS release of Yocto for

- If not yet installed, then refer to earlier notes and Compatible Linux Distribution and Build Host Packages sections of Yocto Project Quick Build guide (<https://www.yoctoproject.org/docs/current/briefyoctoprojectqs/briefyoctoprojectqs.html>).

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

What is real time?

- The nature of real-time programming has a range of contradictory definitions. The author presents what is important about real time.
- A task is a real-time task if it must complete before a certain point in time, known as the deadline.
- The distinction between real-time and non-real-time tasks is shown by considering what happens when an audio stream is playing on the computer while compiling the Linux kernel.
- The first is a real-time task, the compilation is not real-time because there is no deadline .. it needs to complete as soon as possible

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

What is real time?

- The nature of real-time programming has a range of contradictory definitions. The author presents what is important about real time.
- A task is a real-time task if it must complete before a certain point in time, known as the deadline.
- The distinction between real-time and non-real-time tasks is shown by considering what happens when an audio stream is playing on the computer while compiling the Linux kernel.
- The first is a real-time task because there is a constant stream of data arriving at the audio driver, and blocks of audio samples must be written to the audio interface at the playback rate.
- Meanwhile, the compilation is not real-time because there is no deadline. It needs to complete as soon as possible; whether it takes 10 seconds or 10 minutes does not affect the quality of the kernel binaries.
- The other important thing to consider is the consequence of missing the deadline, which can range from mild annoyance through to system failure or, in the most extreme cases, injury or death.
- Some examples:
 - Playing an audio stream: There is a deadline in the order of tens of milliseconds. If the audio buffer underruns, you will hear a click, which is annoying but you will get over it.
 - Moving and clicking mouse: The deadline is also in the order of tens of milliseconds. If missed, mouse moves erratically and button clicks are lost. If the problem persists, the system becomes unusable.
 - Printing a piece of paper: The deadlines for the paper feed are in the millisecond range, which if missed may cause the printer to jam, and somebody will have to go and fix it. Occasional jams are acceptable ...

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

What is real time?

- Printing sell-by dates on bottles on a production line: If one bottle is not printed, the whole production line has to be halted, the bottle removed, and the line restarted, which is expensive.
- Baking a cake: There is a deadline of 30 minutes or so. If you miss it by a few minutes, the cake might be ruined. If you miss it by a lot, the house may burn down.
- A power-surge detection system: If the system detects a surge, a circuit breaker has to be triggered within 2 milliseconds. Failing to do so causes damage to the equipment and may injure or kill someone.
- In other words, there are many consequences to missed deadlines.
- These are the different categories:
 - Soft real-time: The deadline is desirable but is sometimes missed without the system being considered a failure. The first two examples in the previous list are examples of this.
 - Hard real-time: Here, missing a deadline has a serious effect. We can further subdivide hard real-time into mission-critical systems, in which there is a cost to missing the deadline, such as the fourth example, and safety critical systems, in which there is a danger to life and limb, such as the last two examples.
- The author put in the baking example to show that not all hard real-time systems have deadlines measured in milliseconds or microseconds.
- Software written for safety-critical systems has to conform to various standards that seek to ensure that it is capable of performing reliably. It is very difficult for a complex operating system such as Linux to meet those requirements.
- When it comes to mission-critical systems, it is possible, and common, for Linux to be used for a wide range of control systems.
- The requirements of the software depend on the combination of the deadline and the confidence level, which can usually be determined through extensive testing.
- Therefore, to say that a system is real-time, you have to measure its response times under the maximum anticipated load and show that it meets the deadline for an agreed proportion of the time.
- As a rule of thumb, a well-configured Linux system using a mainline kernel is good for soft real-time tasks with deadlines down to tens of milliseconds, and a kernel with the PREEMPT_RT patch is good for soft and hard real-time mission-critical systems with deadlines down to several hundreds of microseconds.
- The key to creating a real-time system is to reduce the variability in response times so that you have greater confidence that the deadlines will not be missed; in other words, you need to make the system more deterministic.
- Often, this is done at the expense of performance. For example, caches make systems run faster by making the average time to access an item of data shorter, but the maximum time is longer in the case of a cache miss.
- Caches make a system faster but less deterministic, which is the opposite of what we want.
- It is a myth of real-time computing that it is fast.
- This is not so; the more deterministic a system is, the lower the maximum throughput.
- The main thing to be concerned with is identifying the causes of latency and the things to do to reduce it.

Identifying sources of nondeterminism

- Fundamentally, real-time programming is about making sure that the threads controlling the output in real time are scheduled when needed and so it can complete the job before the deadline.
- Anything that prevents this is a problem.
- These are some problem areas:
 - Scheduling latency, Priority inversion, Accurate timers, Page faults, Interrupts, Processor caches, Memory bus contention

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Identifying sources of nondeterminism

- Fundamentally, real-time programming is about making sure that the threads controlling the output in real time are scheduled when needed and so it can complete the job before the deadline.
- Anything that prevents this is a problem.
- Here are some problem areas:
- **Scheduling:** Real-time threads must be scheduled before others, and so they must have a real-time policy, SCHED_FIFO or SCHED_RR. Additionally, they should have priorities assigned in descending order, starting with the one with the shortest deadline, according to the theory of rate monotonic analysis.
- **Scheduling latency:** The kernel must be able to reschedule as soon as an event such as an interrupt or timer occurs, and not be subject to unbounded delays. Reducing scheduling latency is a key design factor.
- **Priority inversion:** This is a consequence of priority-based scheduling, which leads to unbounded delays when a high-priority thread is blocked on a mutex held by a low-priority thread. User space has priority inheritance and priority ceiling mutexes; in kernel space, we have RT-mutexes, which implement priority inheritance and will get discussed in the section on the real-time kernel.
- **Accurate timers:** For managing deadlines in the region of low milliseconds or microseconds, it needs timers that match. High-resolution timers are crucial and are a configuration option on almost all kernels.
- **Page faults:** A page fault while executing a critical section of code will upset all timing estimates. Avoid them by locking memory, as described later on.
- **Interrupts:** They occur at unpredictable times and can result in an unexpected processing overhead if there is a sudden flood of them. There are two ways to avoid this. One is to run interrupts as kernel threads, and the other, on multi-core devices, is to shield one or more CPUs from interrupt handling. both are discussed

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Identifying sources of nondeterminism

- **Processor caches:** These provide a buffer between the CPU and the main memory and, like all caches, are a source of non-determinism, especially on multi-core devices. Unfortunately, this is beyond the scope of discussion, but refer to the references at the end of the chapter for more details.
- **Memory bus contention:** When peripherals access memory directly through a DMA channel, they use up a slice of memory bus bandwidth, which slows down access from the CPU core (or cores) and so contributes to nondeterministic execution of the program. However, this is a hardware issue and is also beyond the scope of discussion here.

Understanding scheduling latency

- Real-time threads need to be scheduled as soon as they have something to do.
- However, even if there are no other threads of the same or higher priority, there is always a delay from the point at which the wake-up event occurs—an interrupt or system timer—to the time that the thread starts to run.
- This is called scheduling latency. It consists of
 - hardware interrupt latency
 - interrupt latency
 - preemption latency

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Understanding scheduling latency

- Real-time threads need to be scheduled as soon as they have something to do.
- However, even if there are no other threads of the same or higher priority, there is always a delay from the point at which the wake-up event occurs—an interrupt or system timer—to the time that the thread starts to run.
- This is called **scheduling latency**. It consists of
 - **hardware interrupt latency** from the point at which an interrupt is asserted until the interrupt service routine (ISR) begins to run. A small part of this is the delay in the interrupt hardware itself, but the biggest problem is due to interrupts being disabled in software. Minimizing this IRQ off time is important.
 - **interrupt latency** which is the length of time until the ISR has serviced the interrupt and woken up any threads waiting on this event. It is mostly dependent on the way the ISR was written. Normally, it should take only a short time, measured in microseconds.
 - **preemption latency**, which is the time from the point that the kernel is notified that a thread is ready to run to that at which the scheduler actually runs the thread. It is determined by whether the kernel can be preempted or not. If it is running code in a critical section, then the rescheduling will have to wait. The length of the delay is dependent on the configuration of kernel preemption.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Kernel preemption

- Preemption latency occurs because it is not always safe or desirable to preempt the current thread of execution and call the scheduler.
- Mainline Linux has three settings for preemption, in Kernel Features | Preemption Model menu:
 - CONFIG_PREEMPT_NONE: no preemption.
 - CONFIG_PREEMPT_VOLUNTARY: enables additional checks.
 - CONFIG_PREEMPT: allows the kernel to be preempted.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Kernel preemption

- Preemption latency occurs because it is not always safe or desirable to preempt the current thread of execution and call the scheduler.
- Mainline Linux has three settings for preemption, in Kernel Features | Preemption Model menu:
- CONFIG_PREEMPT_NONE: No preemption
With preemption set to none, kernel code continues without rescheduling until it either returns via a syscall back to user space and be preempted there, or it encounters a sleeping wait that stops the current thread. It reduces the number of transitions between kernel and user space and reduces total number of context switches, therefore this option results in highest throughput at the expense of large preemption latencies. It is the default for servers and desktop kernels where throughput is more important than responsiveness.
- CONFIG_PREEMPT_VOLUNTARY: enable additional checks
This option enables explicit preemption points, where the scheduler is called if the need_resched flag is set. It reduces worst-case preemption latencies at the expense of slightly lower throughput. It is on desktops.
- CONFIG_PREEMPT: This allows the kernel to be preempted.
This option makes the kernel preemptible, meaning that an interrupt can result in an immediate reschedule so long as the kernel is not executing in an atomic context, which is described next. It reduces worst-case preemption latencies and, therefore, overall scheduling latencies to something in the order of a few milliseconds on typical embedded hardware.
It is often described as a soft real-time option, and most embedded kernels are configured in this way. There is a small reduction in overall throughput, but that is usually less important than having more deterministic scheduling for embedded devices.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Kernel preemption

The real-time Linux kernel PREEMPT_RT

- It is a long-standing effort to reduce latencies even further that goes by the name of the kernel configuration option for these features, PREEMPT_RT. The project was started by Ingo Molnar, Thomas Gleixner, and Steven Rostedt and has had contributions from many more developers over the years.
- The kernel patches are at <https://www.kernel.org/pub/linux/kernel/projects/rt>, and there is a wiki at <https://wiki.linuxfoundation.org/realtime/start>. A FAQ, albeit out of date, can also be found at https://rt.wiki.kernel.org/index.php/Frequently_Asked_Questions.
- Many parts of the project are incorporated into mainline Linux over the years, including high-resolution timers, kernel mutexes, and threaded interrupt handlers.
- However, core patches remain outside of the mainline because they are rather intrusive and (some claim) only benefit a small percentage of the total Linux user base. Maybe one day it will all be merged upstream.
- The central plan is to reduce the amount of time the kernel spends running in an atomic context, which is where it is not safe to call the scheduler and switch to a different thread.
- Typical atomic contexts are when the kernel is in the following states:
 - Running an interrupt or trap handler.
 - Holding a spin lock or is in an RCU-critical section. Spin locks and RCU are kernel-locking primitives.
 - Between calls to `preempt_disable()` and `preempt_enable()`.
 - Hardware interrupts are disabled (IRQs off).
- The changes that are part of PREEMPT_RT fall into two main areas: one is to reduce impact of interrupt handlers by turning them into kernel threads, and the other is to make locks preemptible so that a thread can sleep while holding one. It is obvious that there is a large overhead in these changes, which makes average-case interrupt handling slower but much more deterministic, which is the objective here.

Threaded interrupt handlers

- Not all interrupts are triggers for real-time tasks, but all interrupts steal cycles from real-time tasks. Threaded interrupt handlers allow a priority to be associated with the interrupt and for it to be scheduled at an appropriate time
- If the interrupt handler code is run as a kernel thread, there is no reason why it cannot be preempted by a user space thread of higher priority, and so the interrupt handler does not contribute toward scheduling latency of the user space thread.
- Threaded interrupt handlers have been a feature of mainline Linux since 2.6.30. An individual interrupt handler can be threaded by registering it with `request_threaded_irq()` in place of the normal `request_irq()`.
- threaded IRQs are the default with kernel config `CONFIG_IRQ_FORCED_THREADING=y`, which makes all handlers into threads unless they have explicitly prevented this by setting the `IRQF_NO_THREAD` flag.
- When applying the PREEMPT_RT patches, interrupts are, by default, configured as threads in this way.
- Here is an example of what it could show:

```
# ps -Leo pid,tid,class,rtprio,stat,comm,wchan | grep FF
PID TID CLS RTPRIO STAT COMMAND WCHAN
3 3 FF 1 S ksoftirqd/0 smpboot_th
7 7 FF 99 S posixcpu_tmr/0 posix_cpu_
19 19 FF 50 S irq/28-edma irq_thread
20 20 FF 50 S irq/30-edma_err irq_thread
42 42 FF 50 S irq/91-rtc0 irq_thread
43 43 FF 50 S irq/92-rtc0 irq_thread
44 44 FF 50 S irq/80-mmc0 irq_thread
45 45 FF 50 S irq/150-mmc0 irq_thread
47 47 FF 50 S irq/44-mmc1 irq_thread
52 52 FF 50 S irq/86-44e0b000 irq_thread
59 59 FF 50 S irq/52-tilcdc irq_thread
```

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Kernel preemption

```
65 65 FF 50 S irq/56-4a100000 irq_thread
66 66 FF 50 S irq/57-4a100000 irq_thread
67 67 FF 50 S irq/58-4a100000 irq_thread
68 68 FF 50 S irq/59-4a100000 irq_thread
76 76 FF 50 S irq/88-OMAP UAR irq_thread
```

- In this case, which is a BeagleBone running linux-yocto-rt, only the gp_timer interrupt was not threaded.
- It is normal that the timer interrupt handler is run inline.
- The interrupt threads have all been given the default SCHED_FIFO policy and a priority of 50.
- It doesn't make sense to leave them at the defaults, however; assign priorities according to the importance of the interrupts compared to real-time user space threads.
- Here is a suggested order of descending thread priorities:
 - POSIX timers thread, posixcpumr, should always have the highest priority.
 - Hardware interrupts associated with the highest-priority real-time thread.
 - highest-priority real-time thread.
 - Hardware interrupts for the progressively lower-priority real-time threads, followed by thread itself.
 - next highest priority real-time thread.
 - Hardware interrupts for non-real-time interfaces.
 - The soft IRQ daemon, ksoftirqd, which on RT kernels is responsible for running delayed interrupt routines and, prior to Linux 3.6, was responsible for running the network stack, the block I/O layer, and other things. May need to experiment with different priority levels to achieve a balance.
- Change the priorities using the chrt command as part of the bootscrip, using a command such as this:
chrt -f -p 90 `pgrep irq/28-edma`
- The pgrep command is part of procs package.

Preemptible kernel locks

- Making majority of kernel locks preemptible is most intrusive change that PREEMPT_RT makes, and this code remains outside of the mainline kernel.
- The problem occurs with spin locks - used for much of kernel locking.
- A spin lock is a busy-wait mutex that does not require a context switch in the contended case, and so it is very efficient if the lock is held for a short time.
- Ideally, they should be locked for less than the time it would take to reschedule twice.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Preemptible kernel locks

- Making majority of kernel locks preemptible is most intrusive change that PREEMPT_RT makes, and this code remains outside of the mainline kernel.
- The problem occurs with spin locks, which are used for much of the kernel locking.
- A spin lock is a busy-wait mutex that does not require a context switch in the contended case, and so it is very efficient if the lock is held for a short time.
- Ideally, they should be locked for less than the time it would take to reschedule twice.
e.g. threads running on two different CPUs contending same spin lock. CPU 0 gets it first, forcing CPU 1 to spin, waiting until it is unlocked:
- The thread that holds the spin lock cannot be preempted since doing so may make the new thread enter the same code and deadlock when it tries to lock the same spin lock.
- Consequently, in mainline Linux, locking a spin lock disables kernel preemption, creating an atomic context.
- This means that a low-priority thread that holds a spin lock can prevent a high-priority thread from being scheduled, a condition otherwise known as priority inversion.
- The solution adopted by PREEMPT_RT is to replace almost all spin locks with RT-mutexes.
- A mutex is slower than a spin lock, but it is fully preemptible.
Not only that, but RT-mutexes implement priority inheritance and so are not susceptible to priority inversion.
- Knowing what's in the PREEMPT_RT patches - so, how to go about getting them?

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Preemptible kernel locks

Getting the PREEMPT_RT patches

- RT developers do not create patch sets for every kernel version because of porting effort involved. On average, they create patches for every other kernel. most recent kernels supported as of time of writing are: 5.10-rt, 5.9-rt, 5.6-rt, 5.4-rt, 5.2-rt, 5.0-rt, 4.19-rt, 4.18-rt, 4.16-rt, 4.14-rt, 4.13-rt, 4.11-rt
- The patches are available at <https://www.kernel.org/pub/linux/kernel/projects/rt>.
- If using the Yocto Project, there is an rt version of the kernel already.
- Otherwise, it is possible that the kernel already has the PREEMPT_RT patch applied. If not, apply patch.
- Make sure that the PREEMPT_RT patch version and kernel version match exactly; otherwise, you will not be able to apply the patches cleanly.
- Then, apply it in the normal way, as shown in the following command lines.
Then, you will be able to configure the kernel with CONFIG_PREEMPT_RT_FULL:

```
$ cd linux-5.4.93  
$ zcat patch-5.4.93-rt51.patch.gz | patch -p1
```
- There could be a problem, since the RT patch will only apply if when using a compatible mainline kernel. You are probably not, because that is the nature of embedded Linux kernels.
- Therefore, spend some time looking at failed patches and fixing them, and then analyzing the board support for the target and adding any real-time support that is missing.
- If not sure what to do, request support from the kernel vendor and from kernel developer forums.

The Yocto Project and PREEMPT_RT

- The Yocto Project supplies two standard kernel recipes: linux-yocto and the latter having the real-time patches already applied.
- Assuming that the target is supported by the Yocto kernels, just need to select linux-yocto-rt as the preferred kernel and declare that the machine is compatible, for example, by adding lines similar to these to the conf/local.conf:

```
PREFERRED_PROVIDER_virtual/kernel = "linux-yocto-rt"  
COMPATIBLE_MACHINE_beaglebone = "beaglebone"
```

High-resolution timers

- Timer resolution is important for precise timing requirements, which is typical for real-time applications.
- The default timer in Linux is a clock that runs at a configurable rate, typically 100 Hz for embedded systems and 250 Hz for servers and desktops.
- The interval between two timer ticks is known as a jiffy and, in the examples given previously, is 10 milliseconds on an embedded SoC and 4 milliseconds on a server.
- Linux gained more accurate timers from the real-time kernel project in version 2.6.18, and now they are available on all platforms, with a high resolution timer source and device driver for it—which is almost always the case.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

High-resolution timers

- Timer resolution is important for precise timing requirements, which is typical for real-time applications.
- The default timer in Linux is a clock that runs at a configurable rate, typically 100 Hz for embedded systems and 250 Hz for servers and desktops.
- The interval between two timer ticks is known as a jiffy and, in the examples given previously, is 10 milliseconds on an embedded SoC and 4 milliseconds on a server.
- Linux gained more accurate timers from the real-time kernel project in version 2.6.18, and now they are available on all platforms, with a high resolution timer source and device driver for it—which is almost always the case.

Configure the kernel with `CONFIG_HIGH_RES_TIMERS=y`.

- With this enabled, all kernel and user space clocks are accurate to the granularity of underlying hardware. Finding the actual clock granularity is difficult. The obvious answer is the value provided by `clock_getres(2)`, but that always claims a resolution of 1 ns.

High-resolution timers

- cyclicttest tool described in this chapter has an option to analyze times reported by clock to guess resolution:
cyclicttest -R
/dev/cpu_dma_latency set to 0us
WARN: reported clock resolution: 1 nsec
WARN: measured clock resolution approximately: 708 nsec
- look at the kernel log messages for strings like this:
dmesg | grep clock
OMAP clockevent source: timer2 at 24000000 Hz
sched_clock: 32 bits at 24MHz, resolution 41ns, wraps every 178956969942ns
OMAP clocksource: timer1 at 24000000 Hz
Switched to clocksource timer1
- Per the author, these above two methods provide rather different numbers, for which there is no good explanation, but since both are below 1 microsecond, he is happy.
- High-resolution timers measure variations in latency with sufficient accuracy.

Avoiding page faults

- page fault occurs when an application reads or writes to memory that is not committed to physical memory.
- impossible to predict when a page fault occurs, so they are a source of non-determinism for RT topics.
- Fortunately, `mlockall(2)` is a function that commits all memory used by a process and locks it to avoid page faults.
- And, it has two flags:

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Avoiding page faults

- page fault occurs when an application reads or writes to memory that is not committed to physical memory.
- impossible to predict when a page fault occurs, so they are a source of non-determinism for RT topics.
- Fortunately, `mlockall(2)` is a function that commits all memory used by a process and locks it to avoid page faults.
- And, it has two flags:
 - MCL_CURRENT: This locks all pages currently mapped.
 - MCL_FUTURE: This locks pages that are mapped in later.
- usually call `mlockall` during app startup with both flags set to lock all current and future memory mappings.
- MCL_FUTURE is not magic, as there will still be a non-deterministic delay when allocating or freeing heap memory using `malloc()/free()` or `mmap()`.
- Such operations are best done at startup and not in the main control loops.
- Memory allocated on the stack is trickier because it is done automatically, and if a function is called that makes a stack deeper than before, it will encounter more memory-management delays.
- A simple fix is to grow the stack to a size larger than ever needed at startup.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Avoiding page faults

- The code would then look like this:

```
#define MAX_STACK (512*1024)
static void stack_grow (void)
{
    char dummy[MAX_STACK];
    memset(dummy, 0, MAX_STACK);
    return;
}
int main(int argc, char* argv[])
{
    [...]
    stack_grow ();
    mlockall(MCL_CURRENT | MCL_FUTURE);
    [...]
```

- The `stack_grow()` function allocates a large variable on the stack and then zeroes it out to force those pages of memory to be committed to this process.

Interrupt shielding

- Using threaded interrupt handlers helps mitigate interrupt overhead by running some threads at a higher priority than interrupt handlers that do not impact realtime tasks.
- When using a multi-core processor, it can take a different approach and shield one or more cores from processing interrupts completely, allowing them to be dedicated to real-time tasks instead.
- This works either with a normal Linux kernel or a PREEMPT_RT kernel.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Interrupt shielding

- Using threaded interrupt handlers helps mitigate interrupt overhead by running some threads at a higher priority than interrupt handlers that do not impact realtime tasks.
- When using a multi-core processor, it can take a different approach and shield one or more cores from processing interrupts completely, allowing them to be dedicated to real-time tasks instead.
- This works either with a normal Linux kernel or a PREEMPT_RT kernel.
- Achieving this is a question of pinning the real-time threads to one CPU and the interrupt handlers to a different one.
- Set the CPU affinity of a thread or process using the taskset command-line tool, or use the sched_setaffinity(2) and pthread_setaffinity_np(3) functions.
- To set the affinity of an interrupt, first note that there is a subdirectory for each interrupt number in /proc/irq/<IRQ number>.
- The control files for the interrupt are in there, including a CPU mask in smp_affinity.
- Write a bitmask to that file with a bit set for each CPU that is allowed to handle that IRQ.
- Stack growing and interrupt shielding are nifty techniques for improving responsiveness, but how to tell whether they are actually working?

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Measuring scheduling latencies

- All the configuration and tuning is pointless if it cannot be shown that the device meets the deadlines.
- It will need own benchmarks for the final testing, but here are two important measurement tools: cyclicttest and Ftrace.

<https://bootlin.com/doc/training/embedded-linux/embedded-linux-slides.pdf>

Measuring scheduling latencies

- All the configuration and tuning is pointless if it cannot be shown that the device meets the deadlines.
- It will need own benchmarks for the final testing, but here are two important measurement tools: cyclicttest and Ftrace.

cyclicttest

- cyclicttest was originally written by Thomas Gleixner and available on most platforms in rt-tests package.
- When using the Yocto Project, create a target image to include rt-tests by building real-time image recipe:
\$ bitbake core-image-rt
- When using Buildroot, add the BR2_PACKAGE_RT_TESTS package in the Target packages | Debugging, profiling and benchmark | rtests menu.
- cyclicttest measures scheduling latencies by comparing actual time taken for sleeping to requested time.
- If there is no latency, they would all be the same, and reported latency would be 0.
- cyclicttest assumes a timer resolution of less than 1 microsecond.
- It has a large number of command-line options.
- To start with, try running this command as root on the target:
cyclicttest -l 100000 -m -n -p 99
/dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 1.14 1.06 1.00 1/49 320
T: 0 (320) P:99 I:1000 C: 100000 Min: 9 Act: 13 Avg:15 Max: 134

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Measuring scheduling latencies

- The options selected are as follows:
 - l N: This loops N times (the default is unlimited).
 - m: This locks memory with mlockall.
 - n: This uses clock_nanosleep(2) instead of nanosleep(2).
 - p N: This uses the real-time priority N.
- The result line shows the following, reading from left to right:
 - T: 0: This was thread 0, the only thread in this run. set the number of threads with parameter -t. (320): This was PID 320.
 - P: 99: The priority was 99.
 - I: 1000: The interval between loops was 1,000 microseconds. Set the interval with the -i N parameter.
 - C: 100000: The final loop count for this thread was 100,000.
 - Min: 9: The minimum latency was 9 microseconds.
 - Act: 13: The actual latency was 13 microseconds. The actual latency is the most recent latency measurement, which only makes sense if you are watching cyclicttest as it runs.
 - Avg: 15: The average latency was 15 microseconds.
 - Max: 134: The maximum latency was 134 microseconds.
- It was on an idle system running an unmodified linux-yocto kernel as a quick demonstration of the tool.
- To be of real use, run tests over a 24-hour period or longer with load representative of maximum expected.
- Of the numbers produced by cyclicttest, the maximum latency is the most interesting, but it would be nice to get an idea of the spread of the values.
- get that by adding -h <N> to obtain a histogram of samples that are up to N microseconds late.
- Using this technique, the author obtained three traces for the same target board running kernels with no preemption, with standard preemption, and with RT preemption while being loaded with Ethernet traffic from a flood ping.
- The command line was as shown here:

```
# cyclicttest -p 99 -m -n -l 100000 -q -h 500 > cyclicttest.data
```
- Then, he used gnuplot to create three graphs, and the following are the observations:
 - For the output generated with no preemption:
Without preemption, most samples are within 100 microseconds of the deadline, but there are some outliers of up to 500 microseconds, which is pretty much expected.
 - For the output generated with standard preemption:
With standard preemption, the samples are spread out at the lower end, but there is nothing beyond 120 microseconds.
 - For output generated with RT preemption:
The RT kernel is a clear winner because everything is tightly bunched around the 20-microsecond mark, and there is nothing later than 35 microseconds.
- **cyclicttest**, is a standard metric for measuring scheduling latencies.
- However, it cannot help identify and resolve specific problems with kernel latency. To do that, use Ftrace.

Using Ftrace

- The kernel function tracer has tracers to help track down kernel latencies—that is what it was originally written for, after all.
- These tracers capture the trace for the worst-case latency detected during a run, showing the functions that caused the delay.
- The tracers of interest, together with the kernel configuration parameters, are as follows:
 - **irqsoff**: CONFIG_IRQSOFF_TRACER traces code that disables interrupts, recording the worst case.
 - **preemptoff**: CONFIG_PREEMPT_TRACER is similar to irqsoff, but traces the longest time that kernel preemption is disabled (only available on preemptible kernels).
 - **preemptirqsoff**: Combines the previous two traces to record the longest time either irqsoff and/or preemption are disabled for.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Measuring scheduling latencies

- **wakeup**: Traces and records the maximum latency that it takes for highest-priority task to get scheduled after it has been woken up.
- **wakeup_rt**: This is the same as wakeup but only for real-time threads with SCHED_FIFO, SCHED_RR, or SCHED_DEADLINE policies.
- **wakeup_dl**: This is the same but only for deadline-scheduled threads with SCHED_DEADLINE policy.
- Be aware that running Ftrace adds a lot of latency, in the order of tens of milliseconds, every time it captures a new maximum, which Ftrace itself can ignore.
- However, it skews the results of user space tracers such as cyclicttest.
In other words, ignore the results of cyclicttest if run while capturing traces.
- Selecting the tracer is the same as for the function tracer looked at in prior chapter
- as an example of capturing a trace for a maximum period with preemption disabled for a period of 60 secs:

```
# echo preemptoff > /sys/kernel/debug/tracing/current_tracer
# echo 0 > /sys/kernel/debug/tracing/tracing_max_latency
# echo 1 > /sys/kernel/debug/tracing/tracing_on
# sleep 60
# echo 0 > /sys/kernel/debug/tracing/tracing_on
```
- The resulting trace, heavily edited, looks like this:

```
# cat /sys/kernel/debug/tracing/trace
# tracer: preemptoff
#
# preemptoff latency trace v1.1.5 on 3.14.19-yoctostandard
# -----
# latency: 1160 us, #384/384, CPU#0 | (M:preempt VP:0, KP:0, SP:0 HP:0)
# -----
# | task: init-1 (uid:0 nice:0 policy:0 rt_prio:0)
# -----
# => started at: ip_finish_output
# => ended at: __local_bh_enable_ip
#
#
# _-----=> CPU#
# / _-----=> irqsoff
# | / _-----=> need-resched
# || / _-----=> hardirq/softirq
# ||| / _-----=> preempt-depth
# |||| / delay
# cmd pid |||| time | caller
# \ / |||| \ \ /
init-1 0..s. 1us+: ip_finish_output
init-1 0d.s2 27us+: preempt_count_add <-cpdma_chan_submit
init-1 0d.s3 30us+: preempt_count_add <-cpdma_chan_submit
init-1 0d.s4 37us+: preempt_count_sub <-cpdma_chan_submit
[...]
init-1 0d.s2 1152us+: preempt_count_sub <-__local_bh_enable
init-1 0d..2 1155us+: preempt_count_sub <-__local_bh_enable_ip
init-1 0d..1 1158us+: __local_bh_enable_ip
init-1 0d..1 1162us!: trace_preempt_on <-__local_bh_enable_ip
init-1 0d..1 1340us: <stack trace>
```
- Here, it can be seen that the longest period with kernel preemption disabled while running the trace was 1160 microseconds.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

Measuring scheduling latencies

- This simple fact is available by reading `/sys/kernel/debug/tracing/tracing_max_latency`, but the previous trace goes further and provides the sequence of kernel function calls that led up to that measurement.
- The column marked delay shows the point on the trail where each function was called, ending with the call `totrace_preempt_on()` at 1162us, at which point kernel preemption is once again enabled.
- With this information, look back through the call chain and work out whether this is a problem or not.
- The other tracers mentioned work in the same way.

Combining cyclicttest and Ftrace

- If cyclicttest reports unexpectedly long latencies, use the breaktrace option to abort the program and trigger Ftrace to obtain more information.
- invoke breaktrace using `-b<N>` or `--breaktrace=<N>`, where N is the number of microseconds of latency that will trigger the trace.
- select the Ftrace tracer using `-T[tracer name]` or one of the following:
 - C: Context switch
 - E: Event
 - f: Function
 - w: Wakeup
 - W: Wakeup-RT
- For example, this will trigger the Ftrace function tracer when a latency greater than 100 microseconds is measured:
`# cyclicttest -a -t -n -p99 -f -b100`
- These are two complementary tools for debugging latency issues.
- cyclicttest detects the pauses and Ftrace provides the details.

Summary Notes

- Knowing how to use GDB for interactive debugging is a useful tool in the embedded system developer's tool chest. It is a stable, well-documented, and well-known entity.
- It has the ability to debug remotely by placing an agent on the target, be it gdbserver for applications or kgdb for kernel code, and although the default command-line user interface takes a while to get used to, there are many alternative frontends.
- The three mentioned were TUI, DDD, and Visual Studio Code.

1. The term real-time is meaningless unless qualified with a deadline and an acceptable miss rate.
2. With these two pieces of information, determine whether or not Linux is a suitable candidate for OS and, if so, begin to tune the system to meet the requirements.
3. Tuning Linux and application to handle real-time events means making it more deterministic so that the real-time threads can meet their deadlines reliably.
4. Determinism usually comes at the cost of total throughput, so a real-time system is not going to be able to process as much data as a non-real-time system.
5. It is not possible to provide mathematical proof that a complex operating system such as Linux will always meet a given deadline, so the only approach is through extensive testing using tools such as cyclicttest and Ftrace and, more importantly, using own benchmarks for the application.
6. To improve determinism, consider both the application and the kernel.
7. When writing real-time applications, follow the guidelines about scheduling, locking, and memory.
8. The kernel has a large impact on the determinism of a system, and there has been a lot of work on this over the years.
9. Enabling kernel preemption is a good first step.
10. If it is missing deadlines more often than needed, then consider the PREEMPT_RT kernel patches.
11. They can certainly produce low latencies, but the fact that they are not in the mainline yet means that there may be problems integrating them with the vendor kernel for the particular board.

Mastering Embedded Linux Programming By Frank Vasquez, Chris Simmonds (Packt – Publisher)

12. instead, or in addition, embark on the exercise of finding the cause of the latencies using Ftrace and similar tools.
13. This is the end of this dissection of embedded Linux.
 - Being an engineer of embedded systems requires a very wide range of skills, which includes a low-level knowledge of hardware and how the kernel interacts with it.
 - need to be an excellent system engineer being able to configure user applications and tune them to work in an efficient manner.
 - using hardware that is, often, only just capable of carrying out the task.
 - There is a quotation that sums this up: An engineer can do for a dollar what anyone else can do for two.
 - The author hopes that you will be able to achieve this with the information provided.

Further reading

The following resources have further information about the topics introduced in this chapter:

- Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications by Giorgio Buttazzo
- Multicore Application Programming by Darryl Gove

Quiz

Canvas

- 10 Questions
- 45 Mins
- 6 Attempts
- Highest Retained

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

10-24

Quiz

Good Luck!

Assignment

1. None
2. ~~Submit screenshot of install and run.~~

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

10-25

Assignment

1. None for this quarter
2. ~~Submit screenshot of install and run.~~

Review & Questions

- Review
- Q & A

2021

Embedded Linux Design and Programming

Raghav Vinjamuri

10-26

Q&A